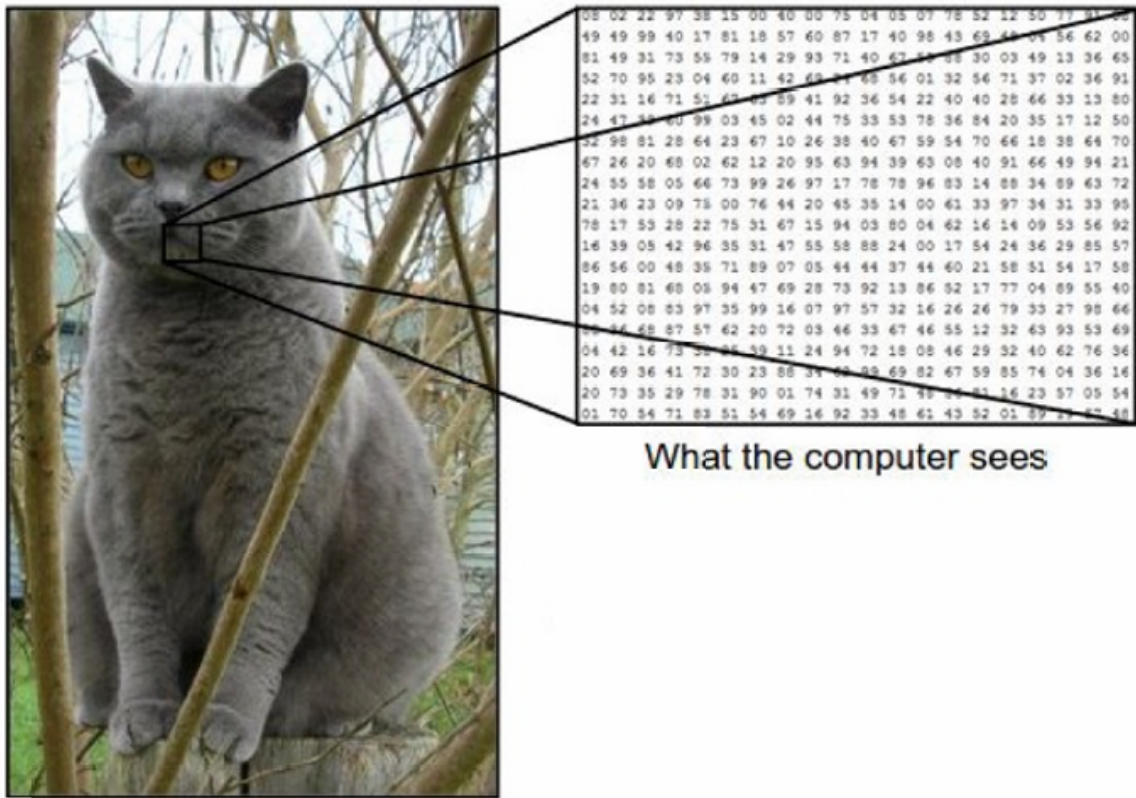


- [1. 语义鸿沟 \(semantic gap\)](#)
- [2. 图像分类器 \(An image classifier\)](#)
- [3. CNN\(Convolutional Neural Network, 卷积神经网络\)](#)
 - [3.1 卷积层 \(Convolution Layer\)](#)
 - [3.1.1 卷积操作细节](#)
 - [3.1.2 卷积层小结](#)
 - [3.2 池化层 \(Pooling layer\)](#)
 - [3.2.1 最大池化 \(Max Pooling\)](#)
 - [3.2.2 小结](#)
 - [3.3 全连接层 \(Fully Connected Layer\)](#)
- [4. LeNet-5](#)
 - [4.1 AlexNet网络架构详细情况](#)
- [5. ZFNet](#)
- [6. VGGNet](#)
- [7. GoogleNet](#)
- [8. ResNet](#)
 - [8.1 残差网络 \(Residual net\)](#)
- [9. 总结](#)

1. 语义鸿沟 (semantic gap)

人类对图像的理解与计算机对图像的理解有所差异。

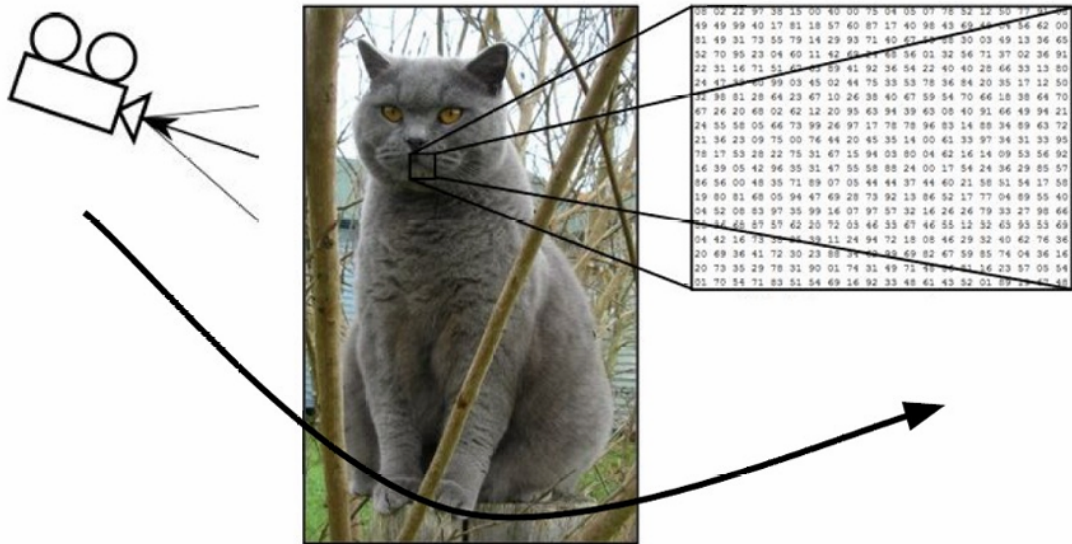
在计算机中，图像通常被表示为三维数组（3D arrays）的数字，这些数字是图像中每个像素的颜色值。对于彩色图像，通常使用RGB（红、绿、蓝）三个颜色通道来表示每个像素的颜色，因此每个像素的颜色值可以由三个数字来表示，这三个数字的范围通常是0到255。



除此之外计算机在识别图片任务上多种挑战:

1. 视角变化 (Viewpoint Variation)

同一个物体从不同的角度或视角拍摄，其外观可能会有很大的不同。人类通常能够很容易地识别出这些不同视角下的猫仍然是同一只猫，但计算机需要通过学习大量的数据和特征来达到类似的识别能力。

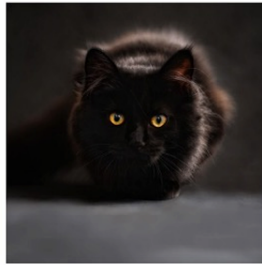


2. 光照变化 (Illumination)

不同的光照条件，如强光、弱光、阴影或逆光，都可能导致图像中物体的外观发生改变。



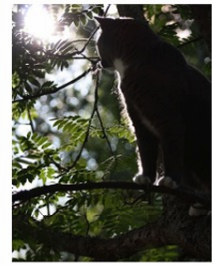
This image is CC0 1.0 public domain



This image is CC0 1.0 public domain



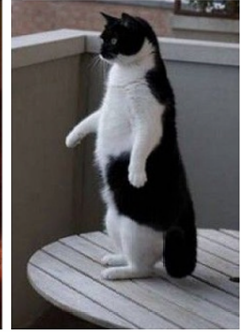
This image is CC0 1.0 public domain



This image is CC0 1.0 public domain

3. 形变 (Deformation)

计算机需要能够识别和理解同一物体在不同形态下的外观。



4. 遮挡 (Occlusion)

计算机需要能够处理和识别被遮挡的物体。



5. 背景杂乱 (Background Clutter)

图像背景中存在许多与目标物体相似或复杂的元素，这使得目标物体难以从背景中区分出来。



6. 类内变化 (Intraclass Variation)

类内变化指的是同一类别 (class) 内不同个体之间的差异。在图像识别任务中，即使是同一类别的

物体，它们在外观上也可能有很大的不同，这种差异可以是颜色、形状、大小、纹理等方面的。



2. 图像分类器 (An image classifier)

与排序数字列表等任务不同，图像分类没有明显的规则或算法可以直接应用。因为前面所说的这些问题都增加了识别的复杂性，没有一种统一的算法可以直接解决这些全部问题。

例如识别图像中的边缘和角点，早期计算机视觉研究中尝试通过分析图像的几何特征来识别图像内容，但它们在处理复杂图像和应对各种变化时存在明显的局限性。

我们现在使用基于数据驱动的机器学习方法来构建图像分类器。步骤如下：

1. 收集数据集：

首先，需要收集一个包含图像及其对应标签的数据集。这些标签是图像中物体的类别名称，例如“猫”、“狗”、“杯子”或“帽子”。

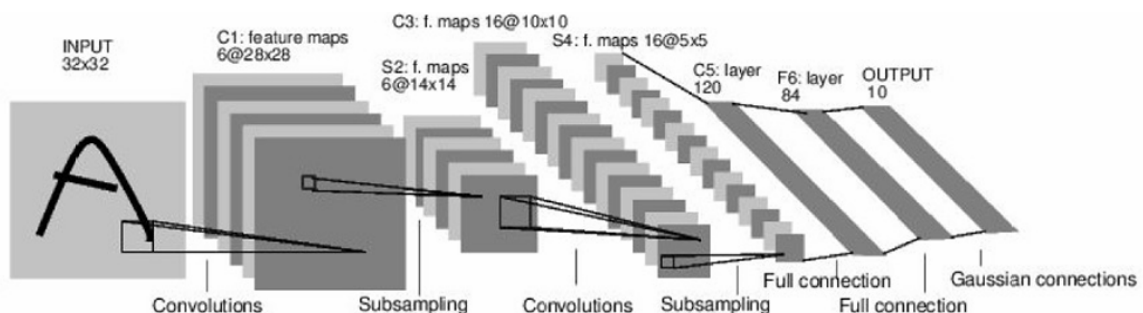
2. 训练图像分类器：

使用机器学习算法来训练一个图像分类器。这个过程涉及使用收集到的数据集来训练模型，使其能够学习如何从图像中提取特征并将其映射到正确的标签上。

3. 评估分类器：

在一个保留的测试图像集上评估分类器的性能。这个测试集是在训练过程中没有使用过的，用于检验模型的泛化能力，即模型在未见过的数据上的表现。

3. CNN(Convolutional Neural Network, 卷积神经网络)



CNN由六个部分组成：

1. 输入层 (Input Layer) :
接收原始图像数据, 通常是一个三维数组, 其维度为[高度, 宽度, 深度 (颜色通道数)]。
2. 卷积层 (Convolutional Layer) :
应用一组可学习的滤波器 (卷积核) 在输入图像上进行卷积操作, 以提取图像的特征。
卷积层可以有多个滤波器, 每个滤波器生成一个特征图 (feature map) 。
3. 池化层 (Pooling Layer) :
进行下采样操作, 减少特征图的空间尺寸, 降低计算量, 同时保留重要特征。
常见的池化操作有最大池化 (Max Pooling) 和平均池化 (Average Pooling) 。
4. 归一化层 (Normalization Layer) :
可选层, 用于对特征图进行归一化处理, 如局部响应归一化 (Local Response Normalization, LRN) 或批归一化 (Batch Normalization) 。
5. 全连接层 (Fully Connected Layer) :
将前面层提取的特征图展平, 并连接到一个或多个全连接层, 这些层负责将特征映射到最终的输出类别。
通常在网络的最后几层使用。
6. 输出层 (Output Layer) :
根据任务的不同, 输出层可以有不同的形式, 如:
分类任务通常使用softmax函数, 输出每个类别的概率。
回归任务可能直接输出一个或多个连续值。

3.1 卷积层 (Convolution Layer)

卷积层通过卷积核在输入图像上滑动并计算点积来提取图像特征的基本过程。

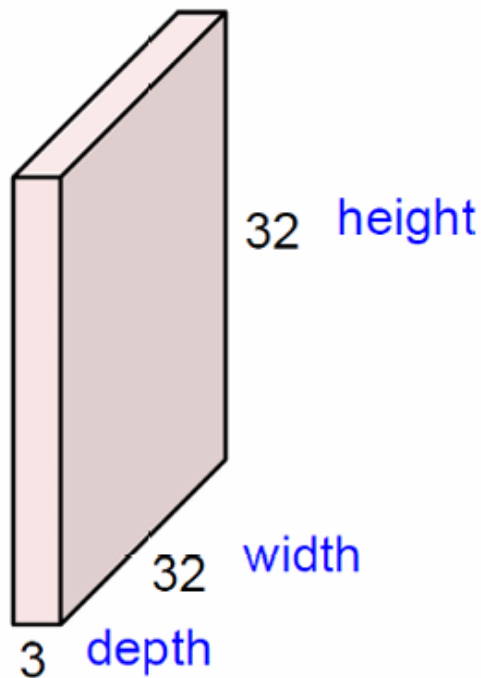
卷积核 (或称为滤波器, filter) 是一个较小的矩阵, 用于在输入图像上滑动并提取特征。在这个例子中, 卷积核的大小是5x5, 深度也是3, 与输入图像的深度相匹配。

卷积操作涉及将卷积核在输入图像上滑动 (即“空间上滑动”), 并在每个位置计算卷积核与图像局部区域的点积 (dot product) 。这个过程可以捕捉图像中的局部特征, 如边缘、纹理等。

点积计算的是卷积核中的权重与图像局部区域像素值的加权和, 这有助于突出图像中的某些特征。

卷积操作的结果是生成一个或多个特征图 (feature maps) , 每个特征图都表示输入图像中不同类型特征的响应。特征图的大小取决于卷积核的大小、步长 (stride) 、填充 (padding) 等因素。

32x32x3 image



5x5x3 filter



卷积核的大小指的是卷积核在图像的宽度和高度上的尺寸。例如，一个3x3的卷积核意味着卷积核在宽度和高度上都是3个像素点。

卷积核的大小决定了它在图像上感受野的大小，即它一次能够“看到”的图像区域的大小。

步长是指卷积核在图像上滑动时的间隔。如果步长为1，卷积核每次移动一个像素点；如果步长大于1，卷积核每次移动多个像素点。

步长影响输出特征图的尺寸。较大的步长会减小输出特征图的尺寸，而较小的步长（如1）会保持或增加输出特征图的尺寸。

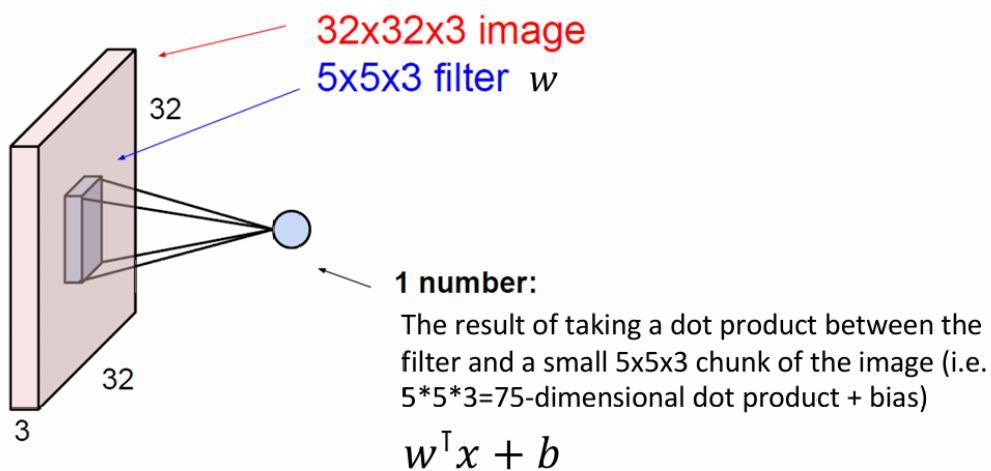
填充是指在输入图像的边缘添加额外的像素点，以控制输出特征图的尺寸。填充通常用0（即黑色）来完成。

填充可以防止卷积操作导致的特征图尺寸缩小过快。例如，使用边缘填充（通常称为“same”填充）可以使输出特征图的尺寸与输入图像相同，如果步长为1的话。

填充的类型包括“valid”（无填充）、“same”（输出尺寸与输入相同）和“full”（输出尺寸最大）。

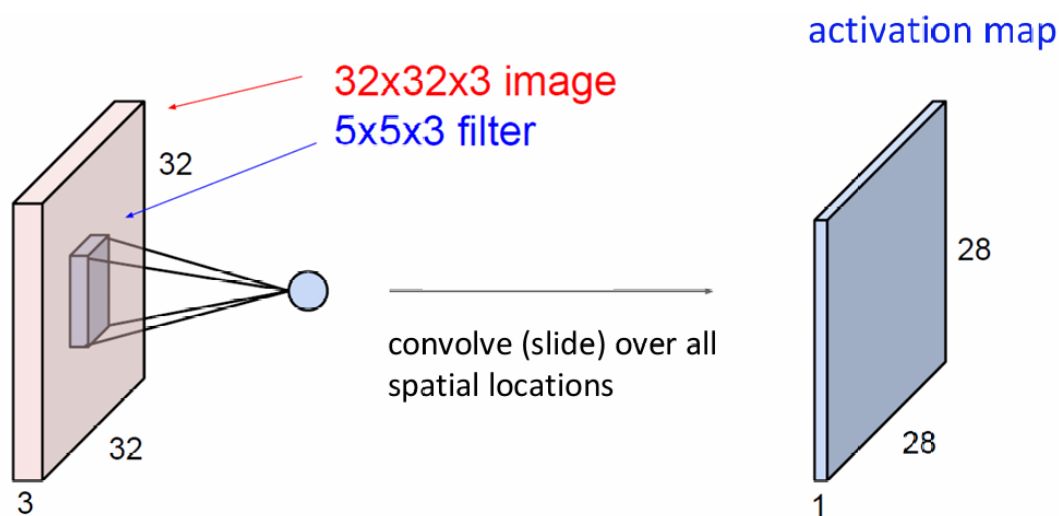
卷积滤波器（也称为卷积核或过滤器）的深度总是与输入数据的深度相匹配（相同）。

例如，这里输入图像是一个RGB图像，那么它通常具有3个颜色通道（红色、绿色和蓝色），因此输入数据的深度是3。在这种情况下，卷积滤波器也会具有3个深度，以便每个颜色通道都可以被独立地处理。

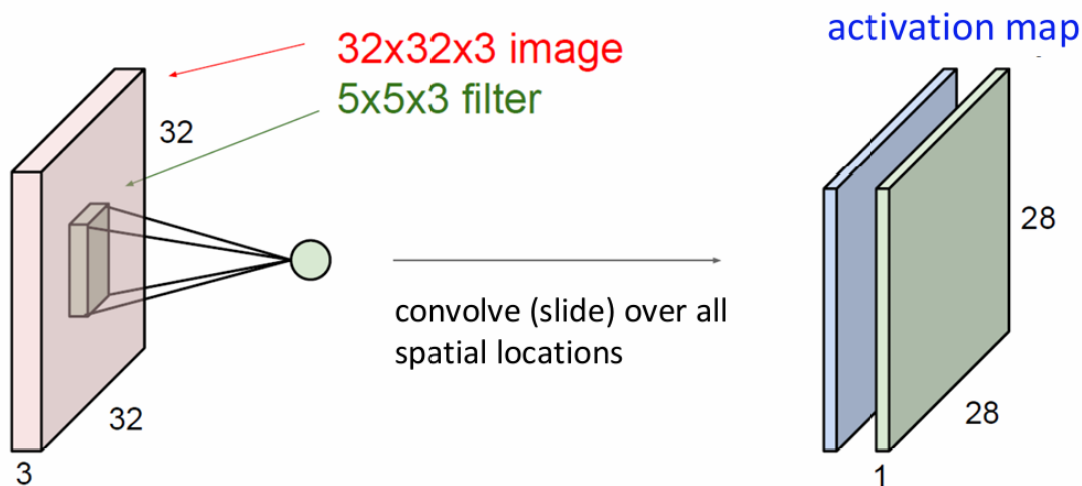


点积计算的是卷积核中的权重与图像局部区域像素值的加权和。

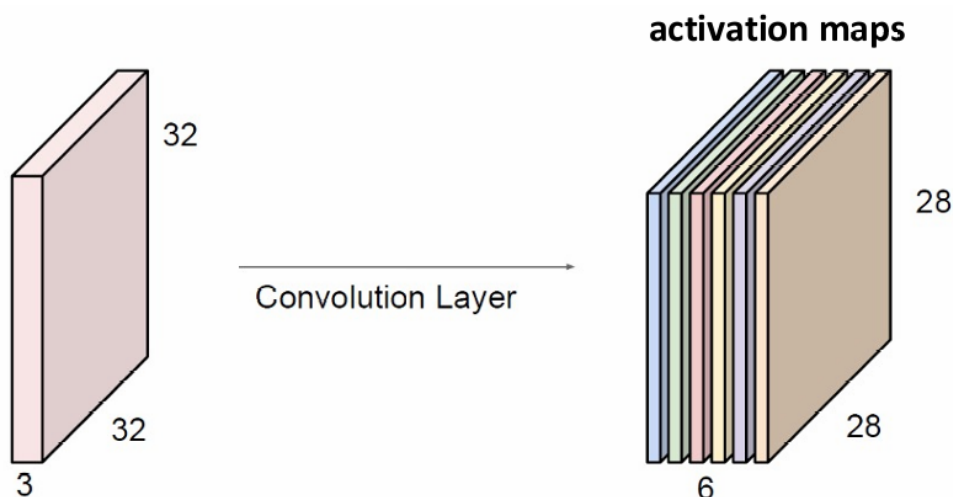
数学表达式为 $w^T x + b$ ，其中 w 是滤波器的权重， x 是图像的局部区域， b 是偏置。



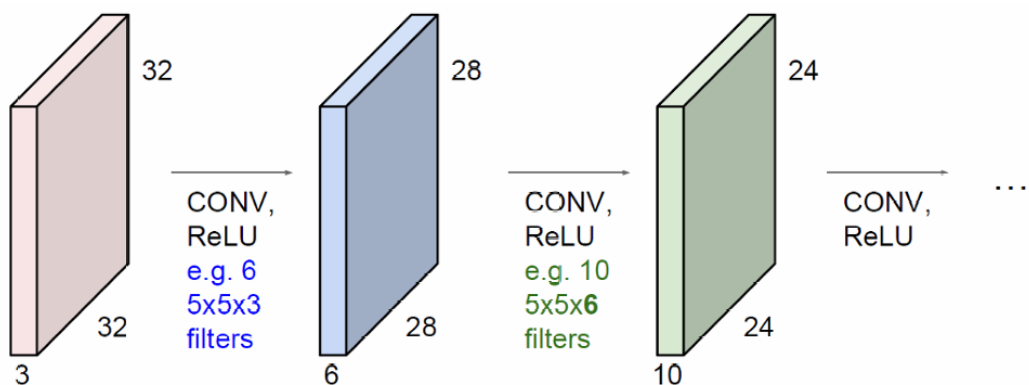
卷积操作会在整个图像上重复进行，生成一个或多个特征图（激活图）。



不同的滤波器生成不同的特征图（激活图）。这些激活图可以堆叠在一起，形成一个三维的激活图集合。



卷积层之后通常会跟一个激活函数，如ReLU（Rectified Linear Unit）。ReLU函数将所有负值置为0，保留正值，这有助于引入非线性，使网络能够学习更复杂的特征。
CNN由一系列卷积层和激活函数组成。



第一个卷积层：

应用卷积操作（CONV）和ReLU激活函数。

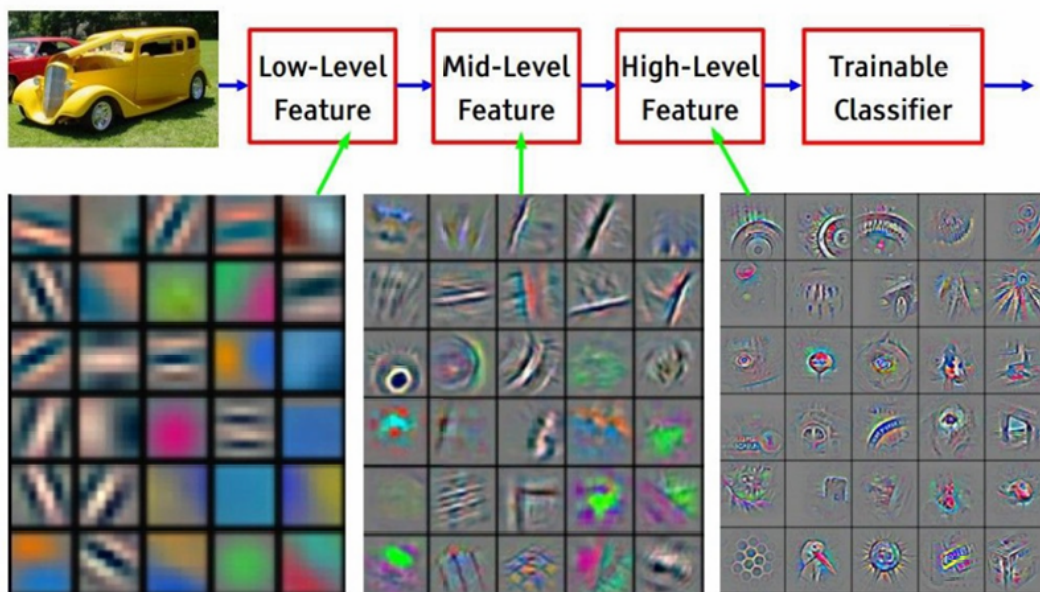
使用6个5x5x3的卷积核（或滤波器）。每个卷积核的尺寸是5x5，深度为3，与输入图像的深度相匹配。输出是6个28x28的特征图（activation maps）。特征图的尺寸从32x32减小到28x28。

第二个卷积层：

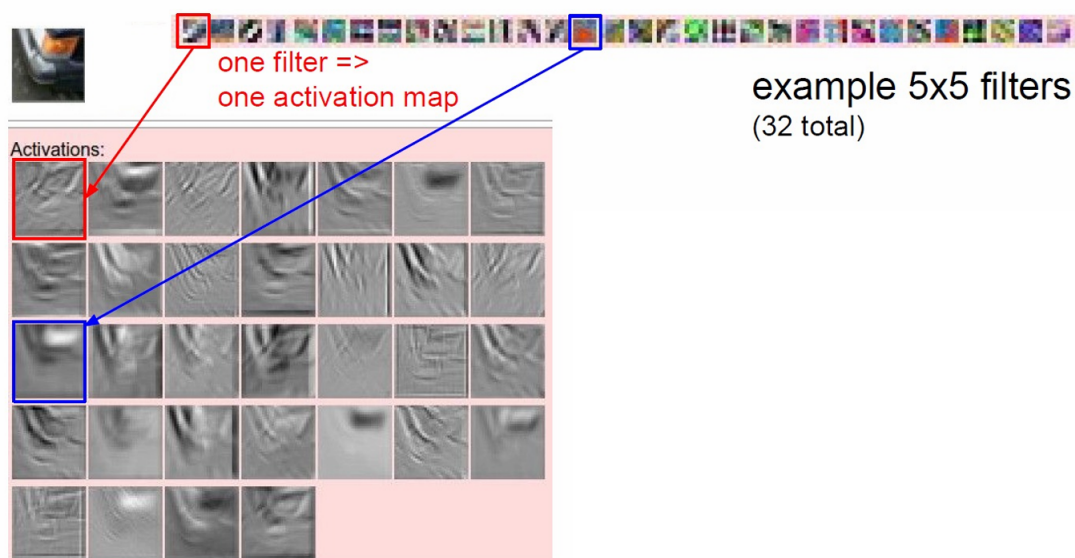
再次应用卷积操作（CONV）和ReLU激活函数。

使用10个5x5x6的卷积核。这里的6指的是前一层输出的深度，即6个特征图。

输出是10个24x24的特征图。特征图的尺寸从28x28减小到24x24。



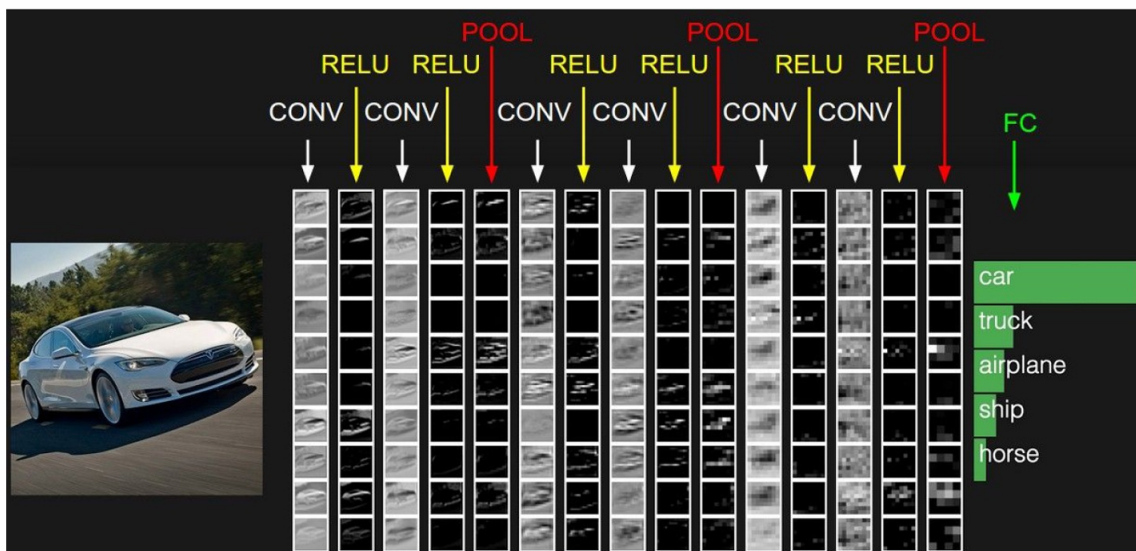
回到整体角度，CNN通过多个卷积层逐步提取从简单到复杂的特征（初始卷积层中提取的低级特征，如边缘和颜色。第二个卷积层提取更复杂的中级特征，如纹理和局部形状。第三个卷积层提取高级特征，这些特征通常对应于图像中的完整对象或对象的部分。），并最终利用这些特征进行图像分类。



为什么这一层被称为卷积层，因为它涉及到两个信号（图像和卷积核）的卷积运算。

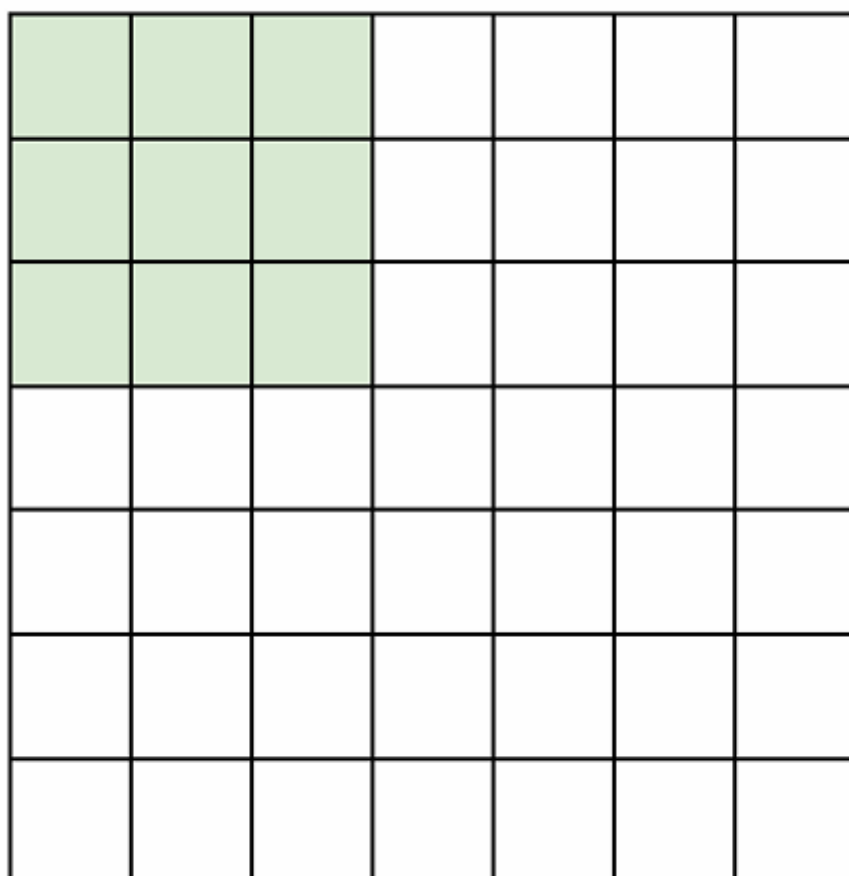
$$f[x, y] * g[x, y] = \sum_{n_1=-\infty}^{\infty} \sum_{n_2=-\infty}^{\infty} f[n_1, n_2] \cdot g[x - n_1, y - n_2]$$

3.1.1 卷积操作细节



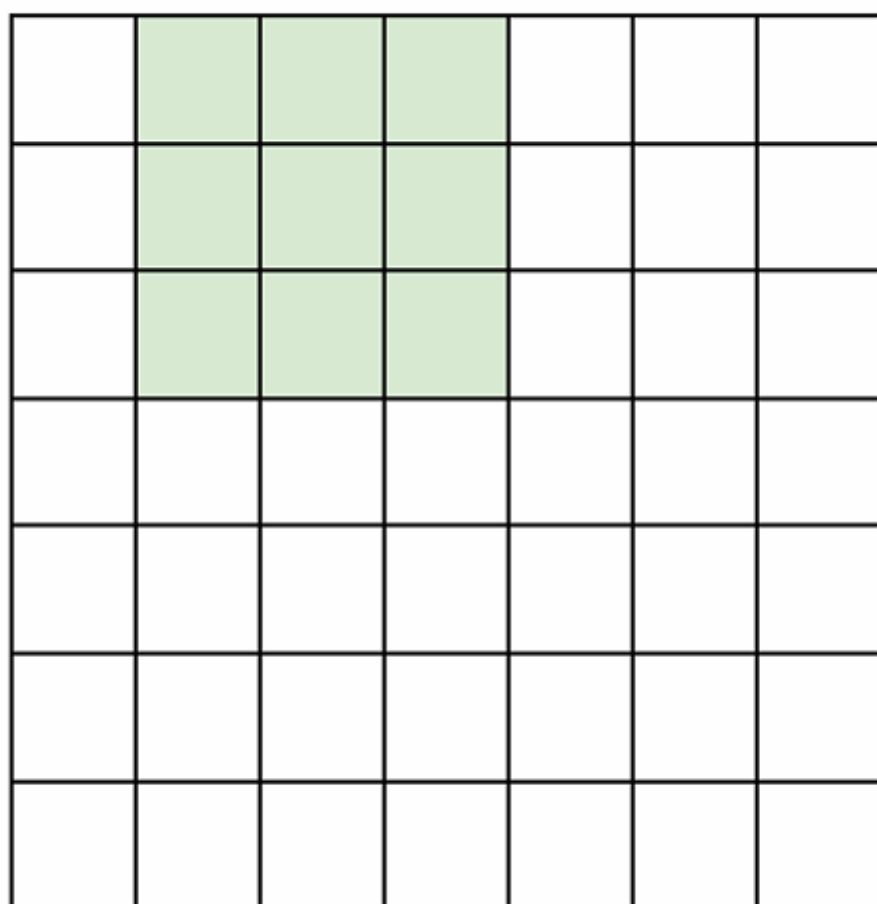
我们刚刚说卷积操作涉及将卷积核在输入图像上滑动（即“空间上滑动”），并在每个位置计算卷积核与图像局部区域的点积（dot product）。
我们来看一下这个操作具体是怎么做的，下面的图片展示了这个过程。

7



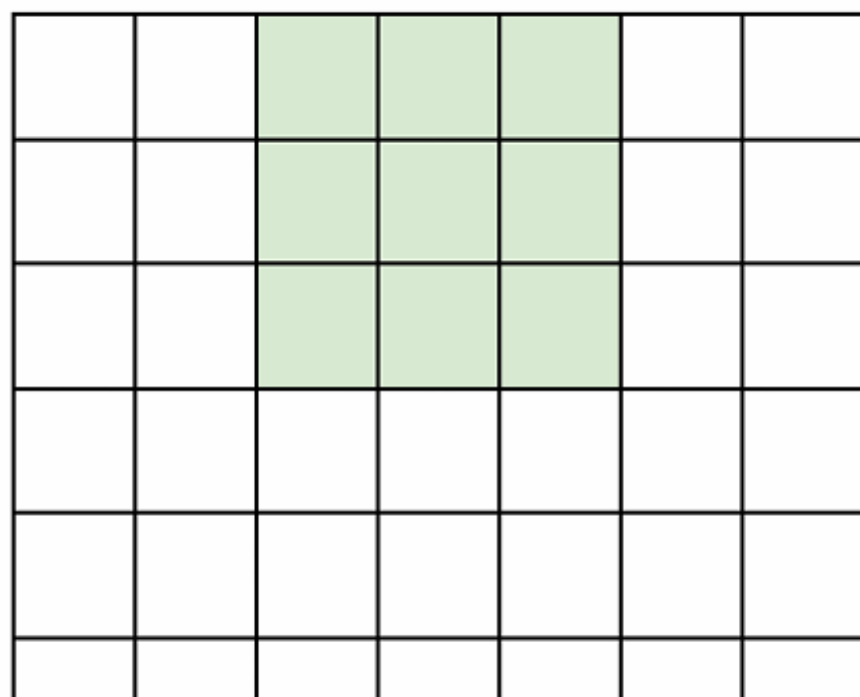
7

7



7

7



7

7

7

7

—

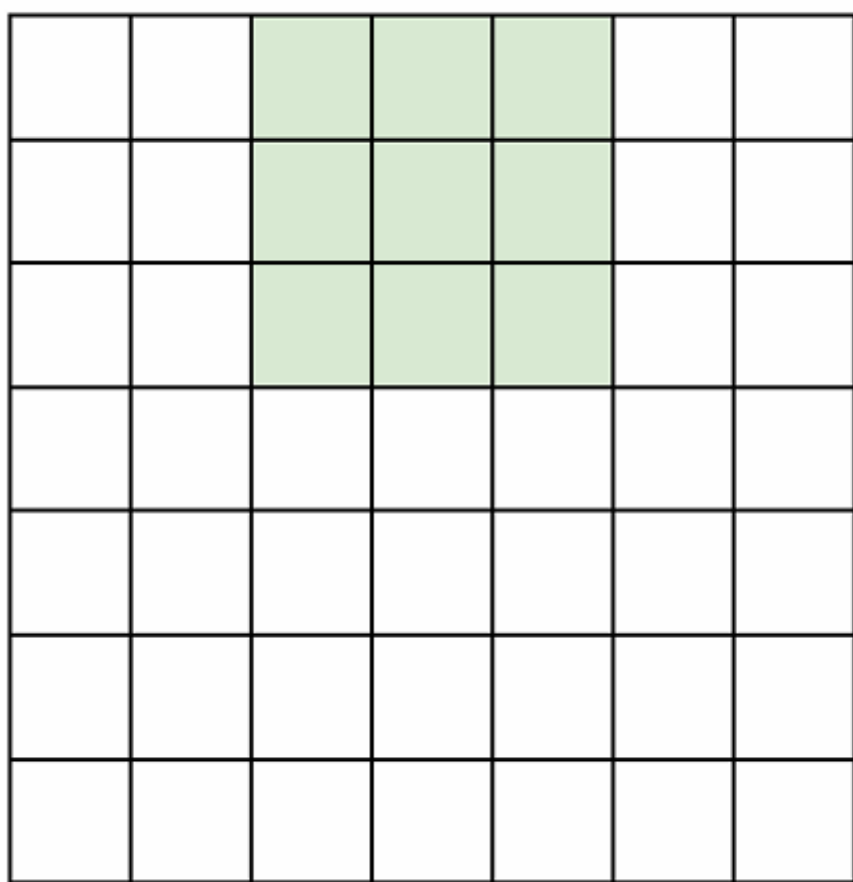
7

上面的图展示了7x7的输入时，用3x3的滤波器，步长为1的滑行过程，最后输出是5x5的特征图。我们现在看步长为2的情况。

7

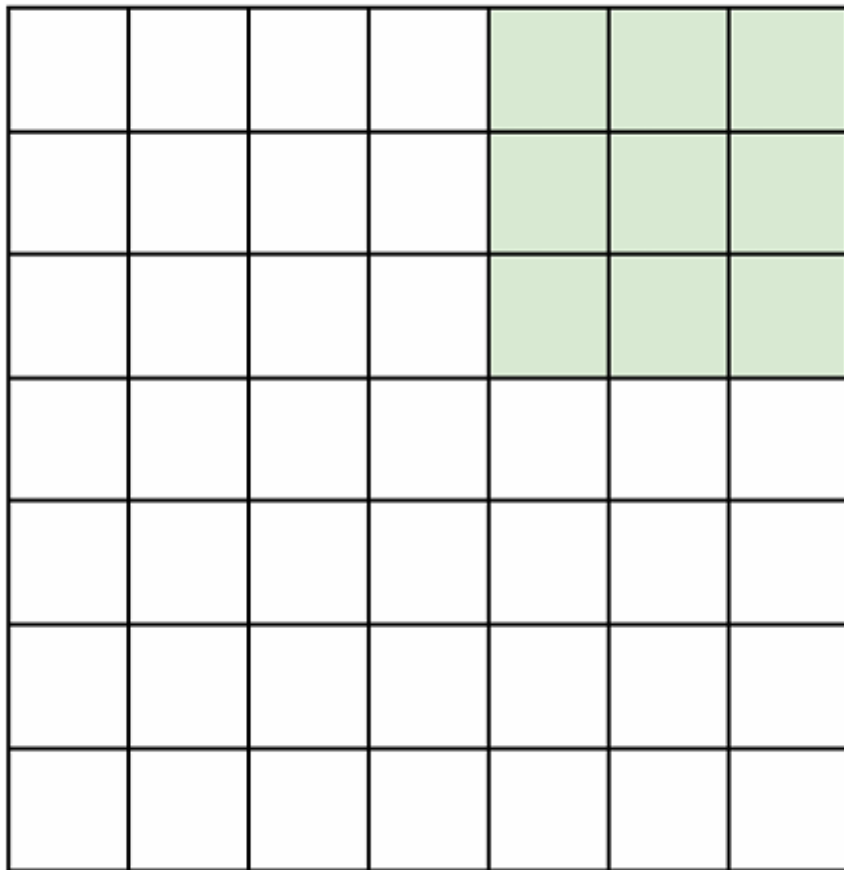
7

7



7

7



7

这时输出是3x3的特征图。

那如果我们想要步长为3呢？

不能应用。这是因为卷积核的大小（3x3）加上步长（3）超过了输入的大小（7x7）。具体来说，卷积核在输入上滑动时，每次移动3个像素，这意味着它只能移动两次（从位置(0,0)到(3,3)，再到(6,6)），而无法覆盖整个7x7的输入区域。

我们现在来总结一下计算输出特征图的尺寸的通用公式。

公式为 $(N - F) / stride + 1$ ，其中 N 是输入尺寸， F 是卷积核尺寸， $stride$ 是步长。

以 $N = 7$ （输入尺寸）， $F = 3$ （卷积核尺寸）为例，计算不同步长下的输出尺寸：

步长1： $(7 - 3) / 1 + 1 = 5$ ，输出尺寸为5x5。

步长2： $(7 - 3) / 2 + 1 = 3$ ，输出尺寸为3x3。

我们再来看下通过零填充（zero padding）来保持输入和输出特征图的空间尺寸一致。

在输入图像的周围添加一圈零值像素，这里添加了1像素的边界。这样做的目的是为了在卷积操作后保持输出特征图的尺寸与输入图像相同。

0	0	0	0	0	0			
0								
0								
0								
0								

根据公式 $(N - F)/stride + 1$ ，计算： $(9 - 3)/1 + 1 = 7$

因此，输出特征图的尺寸也是7x7。

这里的 N 就是原来的输入加上2倍填充量。

这样做保持了卷积操作后输出特征图的尺寸与输入图像相同。

如果卷积核大小为3x3，则添加1像素的零填充；如果卷积核大小为5x5，则添加2像素的零填充；如果卷积核大小为7x7，则添加3像素的零填充。

练习，如果输入时32x32x3，10个5x5卷积核，步长为1，填充2，那么输出的特征图的尺寸是多少？

我们可以根据公式计算 $(32 + 2 * 2 - 5)/1 + 1 = 32$ ，因此输出特征图的尺寸将是32x32x10。

那这一层的参数有多少呢？

每个卷积核有 $5 \times 5 \times 3 + 1 = 76$ 个参数（5x5是卷积核的尺寸，3是输入的深度，+1是偏置），10个卷积核所以是 $76 \times 10 = 760$ 。

3.1.2 卷积层小结

卷积层接受一个尺寸为 $W_1 \times H_1 \times D_1$ 的输入体积，其中 W_1 是宽度， H_1 是高度， D_1 是深度（或通道数）。

需要设置四个超参数：

滤波器数量（Number of filters K ）：卷积层中使用的滤波器数量。

滤波器的空间范围（Spatial extent F ）：每个滤波器的大小，例如3x3、5x5等。

步长（Stride S ）：滤波器在输入体积上滑动的间隔。

零填充量（Amount of zero padding P ）：在输入体积边缘添加的零值像素的数量。

卷积层产生一个尺寸为 $W_2 \times H_2 \times D_2$ 的输出体积，其中：

$$W_2 = \left\lfloor \frac{S \times W_1 - F + 2P}{S} \right\rfloor + 1: \text{输出体积的宽度。}$$

$$H_2 = \left\lfloor \frac{S \times H_1 - F + 2P}{S} \right\rfloor + 1: \text{输出体积的高度。宽度和高度通过对称计算（即考虑填充）。}$$

参数数量：

每个滤波器引入 $F \cdot F \cdot D_1$ 个权重，加上一个偏置 (bias)，总共 $F \cdot F \cdot D_1 + 1$ 个参数。

对于 K 个滤波器，总共有 $(F \cdot F \cdot D_1) \cdot K$ 个权重和 K 个偏置。

$D_2 = K$ ：输出体积的深度等于滤波器的数量。

输出体积的计算：

在输出体积中，第 d 个深度切片（尺寸为 $W_2 \times H_2$ ）是通过在输入体积上执行第 d 个滤波器的有效卷积操作得到的，步长为 S ，然后加上第 d 个偏置。

常见的超参数设置：

$K = (2 \text{ 的幂, 例如 } 32, 64, 128, 512)$ ：滤波器数量通常是2的幂。

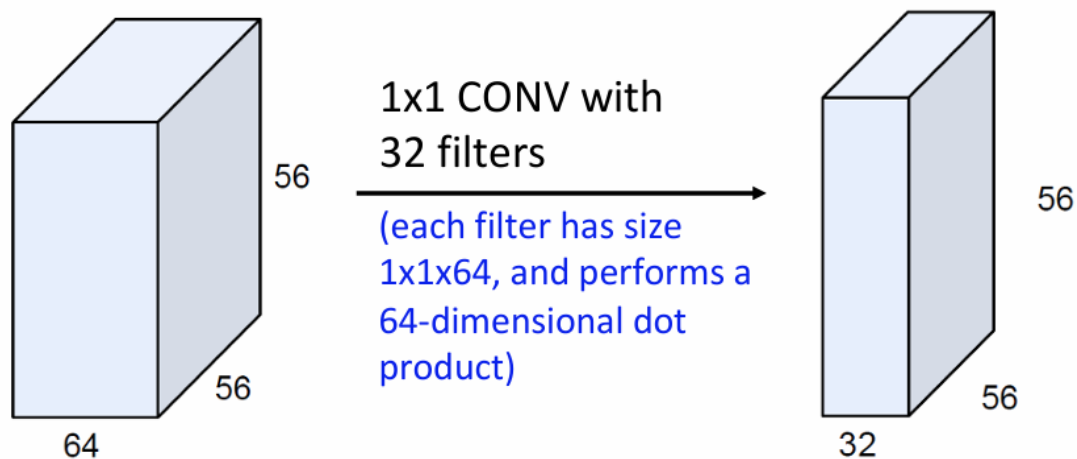
$F = 3, S = 1, P = 1$ ：滤波器大小为 3×3 ，步长为1，填充为1。

$F = 5, S = 1, P = 2$ ：滤波器大小为 5×5 ，步长为1，填充为2。

$F = 5, S = 2, P = ?$ （根据需要调整）：滤波器大小为 5×5 ，步长为2，填充量根据需要调整。

$F = 1, S = 1, P = 0$ ：滤波器大小为 1×1 ，步长为1，无填充。

注意： 1×1 卷积层是一种特殊的卷积层，它使用尺寸为 1×1 的卷积核，通常用于改变输入特征图的深度（即通道数），而不改变其空间维度（宽度和高度）。

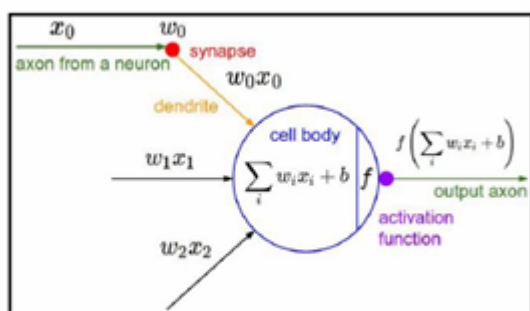
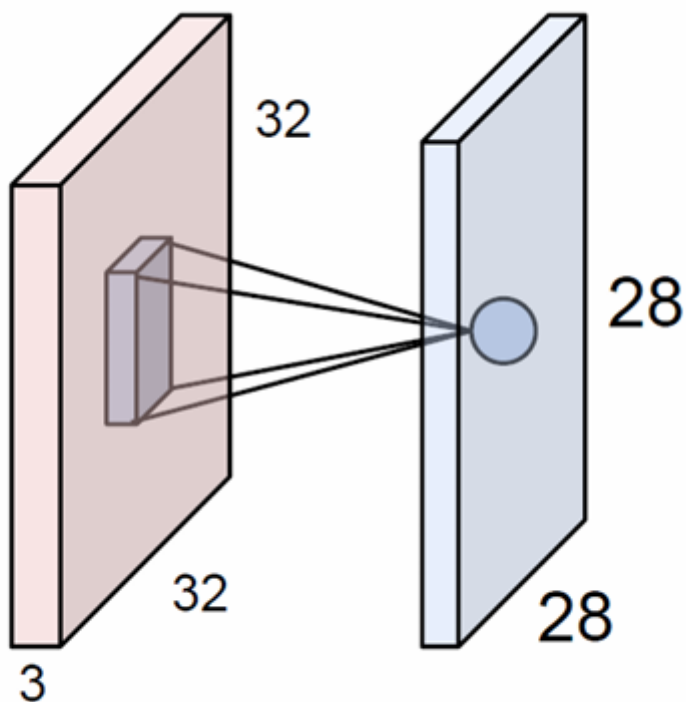


我们使用PyTorch和Caffe都可以使用内置的函数实现卷积层。

既然叫神经网络，我们可以从传统的神经元角度去理解。

例如我们有一个神经网络，我们可以理解为输入（树突，dendrites）、细胞体（cell body）和输出（轴突，output axon）。

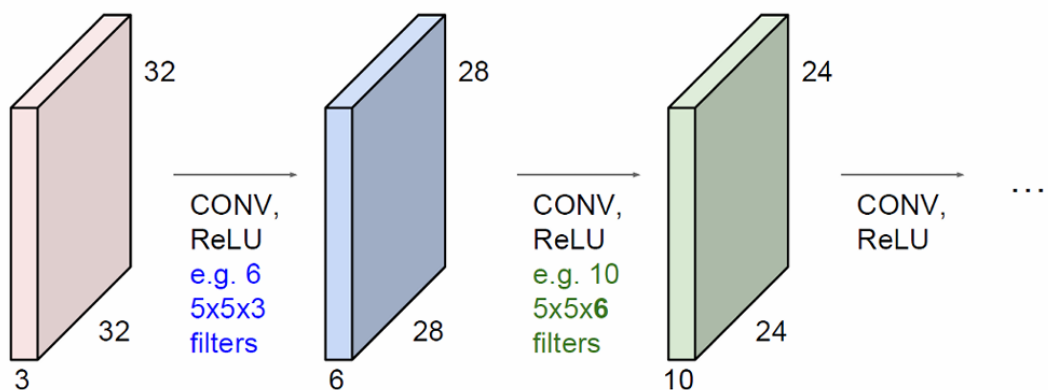
神经元通过加权求和（即点积）其输入信号，并加上一个偏置 (bias)，然后通过激活函数 (activation function) 产生输出。



所有这些神经元共享相同的卷积核参数。这是CNN中参数共享的概念，即相同的卷积核参数在整个输入特征图上滑动，从而减少模型的参数数量并提高计算效率。

每个神经元的感受野是5x5，即卷积核覆盖的区域大小。

如果有多个卷积核，那就是有多个不同的神经元。

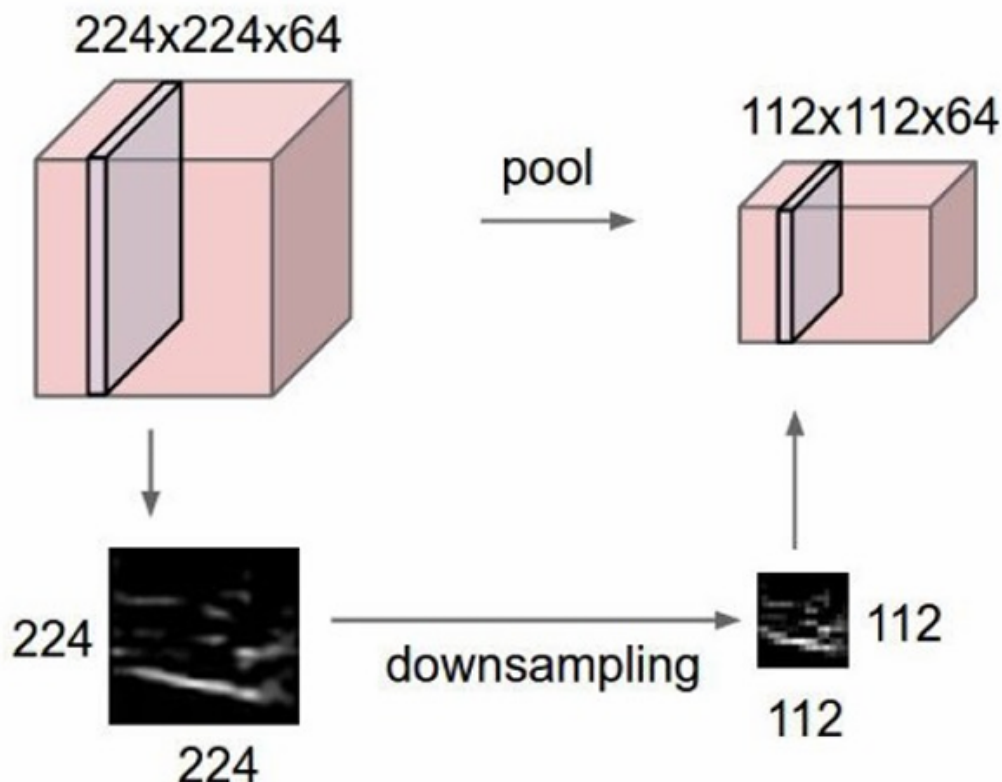


注意如果特征图的尺寸缩小得太快，可能会丢失重要的特征信息，影响模型的性能。

设计CNN时，需要平衡特征图尺寸的缩小速度，以确保既不丢失重要信息，又能有效地提取和学习图像特征。通常，会在卷积层之间加入池化层（pooling layers）来控制特征图的尺寸，同时使用填充（padding）来保持尺寸或减小尺寸缩小的速度。

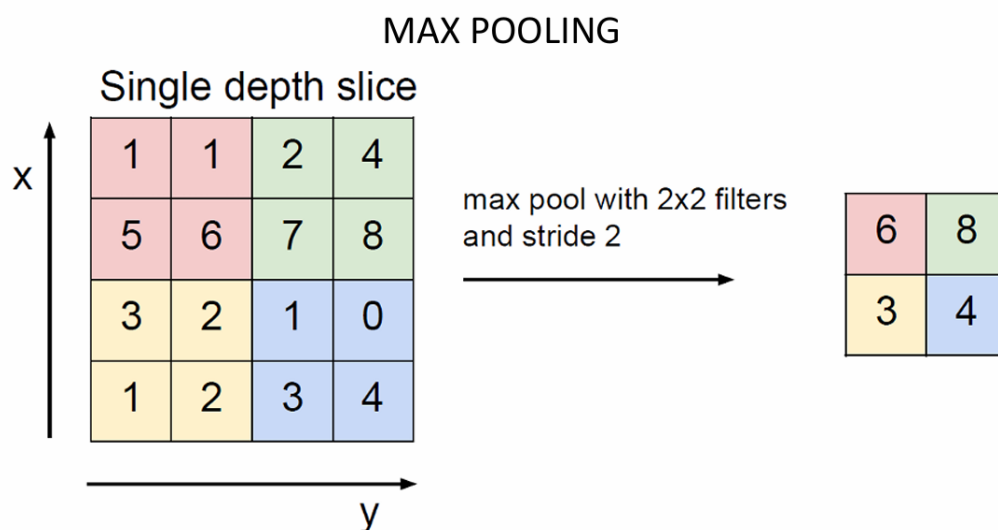
3.2 池化层 (Pooling layer)

池化层通过下采样 (downsampling) 操作, 使特征表示变得更小、更易于管理。它独立地对每个激活图 (activation map) 进行操作, 通常不会改变特征图的深度。如下图所示。



池化层通过下采样技术将特征图的空间尺寸减半, 但深度保持不变。那这样的操作是怎么做到的呢?

3.2.1 最大池化 (Max Pooling)



如图所示, 使用 2×2 的最大池化滤波器 (filter) 和步长 (stride) 为 2 进行最大池化操作。滤波器在特征图上滑动, 每次移动 2 个单位, 选择每个 2×2 区域内的最大值作为输出。因此前面的例子中, 池化窗口大小为 2×2 , 步长为 2, 这才导致输出特征图的空间尺寸为输入尺寸的一

半。

3.2.2 小结

池化层接受一个尺寸为 $W_1 \times H_1 \times D_1$ 的输入体积，其中 W_1 是宽度， H_1 是高度， D_1 是深度（或通道数）。

它产生一个尺寸为 $W_2 \times H_2 \times D_2$ 的输出体积，其中： $D_2 = D_1$ ，即深度保持不变。

需要设置三个超参数：

空间范围（Spatial extent F ）：池化窗口的大小，例如2x2或3x3。

步长（Stride S ）：池化窗口移动的间隔。

池化层通常不涉及零填充（Zero padding），所以没有 P 参数。

输出尺寸的计算：

$W_2 = \left\lfloor \frac{W_1 - F}{S} \right\rfloor + 1$ ：输出体积的宽度。

$H_2 = \left\lfloor \frac{H_1 - F}{S} \right\rfloor + 1$ ：输出体积的高度。

常见的池化设置：

$F = 2, S = 2$ ：使用2x2的池化窗口，步长为2。

$F = 3, S = 2$ ：使用3x3的池化窗口，步长为2。

池化层不引入任何参数，因为它计算输入的固定函数（例如最大值或平均值）。

对于池化层，通常不使用零填充，因为池化操作本身已经减少了特征图的尺寸。

3.3 全连接层（Fully Connected Layer）

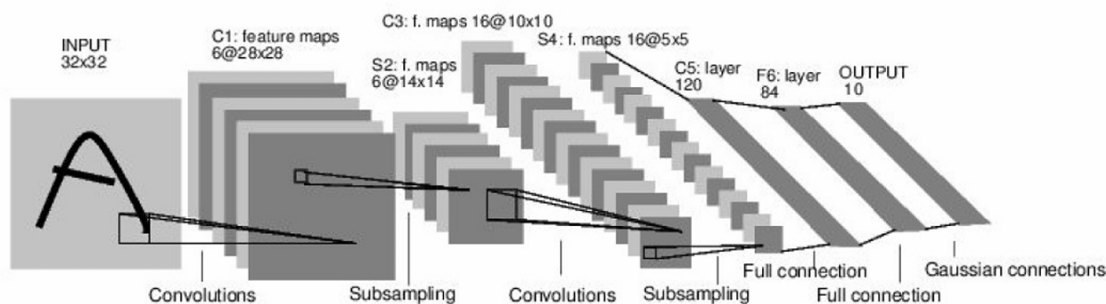
全连接层是CNN中的一层，其中每个神经元都与前一层的所有神经元相连。这意味着每个神经元都会接收前一层所有特征图的输入。

在全连接层中，每个神经元的输入是前一层输出的展平（flattened）版本。展平意味着将前一层的多维特征图（例如，宽度x高度x深度）转换为一维向量，以便每个神经元都可以接收完整的特征信息。

全连接层的工作方式与传统的前馈神经网络中的层类似，其中每个神经元都与前一层的所有神经元相连。这种结构允许网络学习输入特征之间的复杂关系。

4. LeNet-5

LeNet-5是著名的卷积神经网络（CNN）架构之一，由LeCun等人在1998年提出。LeNet-5是早期用于手写数字识别的网络，它在MNIST数据集上取得了很好的效果。



输入层：

输入是一个32x32的灰度图像。

卷积层（C1, C3, S2, S4）：

C1：第一个卷积层，使用6个5x5的卷积核，步长为1，生成6个28x28的特征图。

C3：第二个卷积层，使用16个5x5的卷积核，步长为1，生成16个10x10的特征图。

S2：第一个池化层，使用2x2的最大池化，步长为2，生成6个14x14的特征图。

S4: 第二个池化层, 同样使用2x2的最大池化, 步长为2, 生成16个5x5的特征图。

全连接层 (C5, F6) :

C5: 第一个全连接层, 包含120个神经元。

F6: 第二个全连接层, 包含84个神经元。

输出层:

最后一层是一个具有10个神经元的全连接层, 对应于10个类别 (数字0到9) 的输出。

架构总结:

LeNet-5的架构遵循了卷积-池化-卷积-池化-全连接 (CONV-POOL-CONV-POOL-FC)的结构。

这种结构有效地提取了图像的局部特征, 并通过池化层减少了特征图的空间尺寸, 从而降低了计算复杂度。

那我们借这个案例复习一下本章的知识。

问题1: 如果输入图像的尺寸是 $227 \times 227 \times 3$ (宽x高x通道数)。使用96个 11×11 的卷积核, 步长 (stride) 为4。输出体积尺寸是多少?

对于宽度和高度: $(227 - 11) / 4 + 1 = 56$ 。

因此, 输出特征图的尺寸是 $56 \times 56 \times 96$ 。

问题2: 此时这一层的总参数数量是多少?

每个卷积核有 $11 \times 11 \times 3$ 个权重 (因为输入有3个通道), 加上一个偏置 (bias), 所以每个卷积核有 $11 \times 11 \times 3 + 1 = 364$ 个参数。

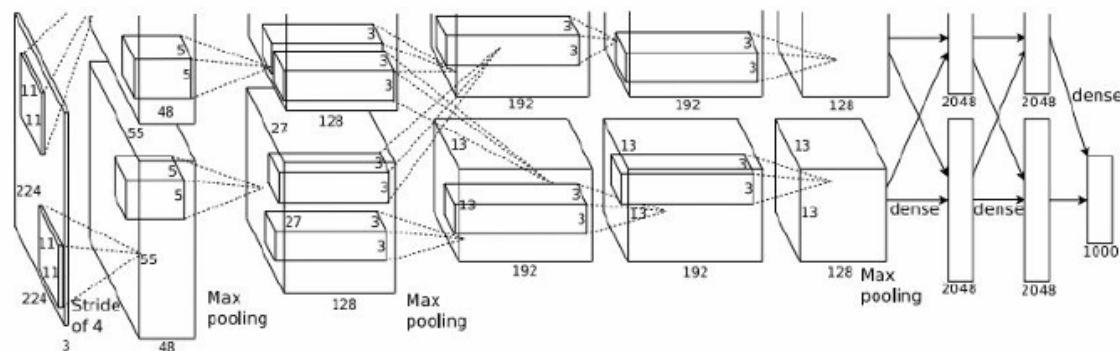
总共有96个这样的卷积核, 所以总参数数量是 $364 \times 96 = 35,904$ 。

问题3: 经过第一层卷积层处理后, 输出特征图的尺寸变为 $55 \times 55 \times 96$ 。使用 3×3 的池化滤波器, 步长 (stride) 为2。这一层的参数数量是多少?

参数是0, 因为池化层不包含可训练的参数。

4.1 AlexNet网络架构详细情况

我们再看一下AlexNet网络架构的详细情况。



卷积层 (CONV1-CONV5) :

CONV1: 使用96个 11×11 的卷积核, 步长为4, 没有填充 (pad 0), 输出特征图尺寸为 $55 \times 55 \times 96$ 。

POOL1: 使用 3×3 的最大池化 (Max pooling), 步长为2, 输出特征图尺寸为 $27 \times 27 \times 96$ 。

NORM1: 归一化层, 用于标准化特征图。

CONV2: 使用256个 5×5 的卷积核, 步长为1, 填充为2 (pad 2), 输出特征图尺寸为 $27 \times 27 \times 256$ 。

POOL2: 使用 3×3 的最大池化, 步长为2, 输出特征图尺寸为 $13 \times 13 \times 256$ 。

NORM2: 归一化层。

CONV3: 使用384个 3×3 的卷积核, 步长为1, 填充为1 (pad 1), 输出特征图尺寸为 $13 \times 13 \times 384$ 。

CONV4: 使用384个 3×3 的卷积核, 步长为1, 填充为1 (pad 1), 输出特征图尺寸为 $13 \times 13 \times 384$ 。

CONV5: 使用256个 3×3 的卷积核, 步长为1, 填充为1 (pad 1), 输出特征图尺寸为 $13 \times 13 \times 256$ 。

POOL3: 使用 3×3 的最大池化, 步长为2, 输出特征图尺寸为 $6 \times 6 \times 256$ 。

全连接层 (FC6-FC8) :

FC6: 4096个神经元的全连接层。

FC7: 4096个神经元的全连接层。

细节:

使用了归一化层（Norm layers），这在当时并不常见。

进行了大量数据增强 (Heavy data augmentation) 。

使用了Dropout 0.5来防止过拟合。

批量大小 (Batch size) 为128。

使用了SGD动量 (SGD Momentum) 0.9。

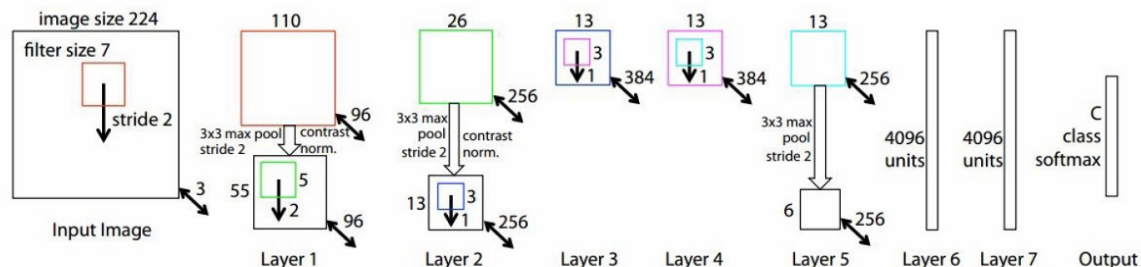
学习率 (Learning rate) 初始为 $1e-2$, 每10个epoch减少10倍。

当验证准确率 (val accuracy) 稳定时手动调整学习率。

L2权重衰减 (L2 weight decay) $5e-4$ 。

使用了7个CNN集成 (ensemble)来提高性能, 前5错误率从18.2%降低到15.4%。

ZFNet网络架构，这是对AlexNet架构的一个变体，由Zeiler和Fergus在2013年提出。ZFNet在ImageNet数据集上取得了比AlexNet更好的性能。



ZFNet相对于AlexNet的改进:

CONV1: 从AlexNet中的11x11卷积核和步长4改为7x7卷积核和步长2。

CONV3,4,5: 在AlexNet中使用384, 384, 256个滤波器, 而在ZFNet中使用512, 1024, 512个滤波器。

这些改进使得ZFNet在ImageNet数据集上的前5错误率从15.4%降低到14.8%。

6. VGGNet

VGGNet是由Simonyan和Zisserman在2014年提出的一种深度卷积神经网络（CNN）架构。

ConvNet Configuration					
A	A-LRN	B	C	D	E
11 weight layers	11 weight layers	13 weight layers	16 weight layers	16 weight layers	19 weight layers
input (224 × 224 RGB image)					
conv3-64	conv3-64 LRN	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64	conv3-64 conv3-64
maxpool					
conv3-128	conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128	conv3-128 conv3-128
maxpool					
conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256	conv3-256 conv3-256 conv1-256	conv3-256 conv3-256 conv3-256	conv3-256 conv3-256 conv3-256 conv3-256
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512	conv3-512 conv3-512 conv1-512	conv3-512 conv3-512 conv3-512	conv3-512 conv3-512 conv3-512 conv3-512
maxpool					
FC-4096					
FC-4096					
FC-1000					
soft-max					

Table 2: Number of parameters (in millions).

Network	A,A-LRN	B	C	D	E
Number of parameters	133	133	134	138	144

VGGNet只使用3x3的卷积核，步长为1，填充为1（pad 1）。

使用2x2的最大池化（Max Pooling），步长为2。

图中用红色框标出了VGGNet的“最佳模型”配置，即配置E。

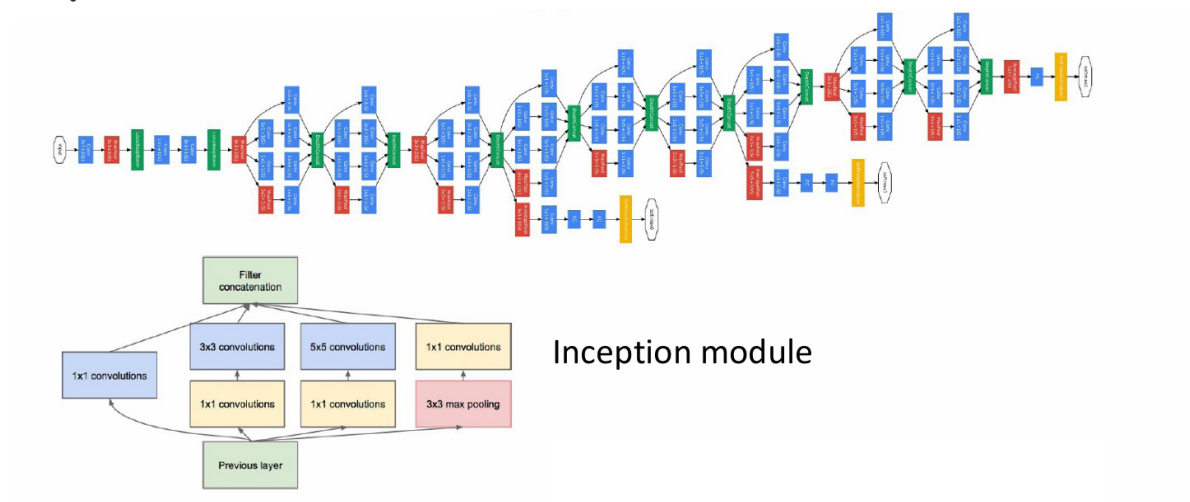
配置E包含19个权重层，是所有配置中层数最多的。

在ILSVRC 2013（ImageNet Large Scale Visual Recognition Challenge 2013）数据集上，最佳模型的前5错误率为11.2%。

进一步优化后，错误率可以降低到7.3%。

7. GoogleNet

这是由Szegedy等人在2014年提出的一种深度卷积神经网络（CNN）架构。GoogLeNet在ILSVRC 2014（ImageNet Large Scale Visual Recognition Challenge）比赛中取得了优异的成绩，其top-5错误率为6.7%。



GoogLeNet由多个Inception模块堆叠而成，每个模块后通常跟随一个池化层。Inception模块通过并行执行不同大小的卷积操作来捕获图像中的多尺度特征。模块包括1x1、3x3和5x5的卷积操作，以及一个3x3的最大池化操作。这些操作的输出被连接（concatenated）在一起，形成模块的输出。

GoogLeNet的一个重要特性是它只有500万个参数，这比当时的其他网络少得多。它完全去除了全连接层（FC layers），通过使用Inception模块来减少参数数量。与AlexNet的比较：

GoogLeNet的参数数量比AlexNet少12倍。

计算量是AlexNet的两倍。

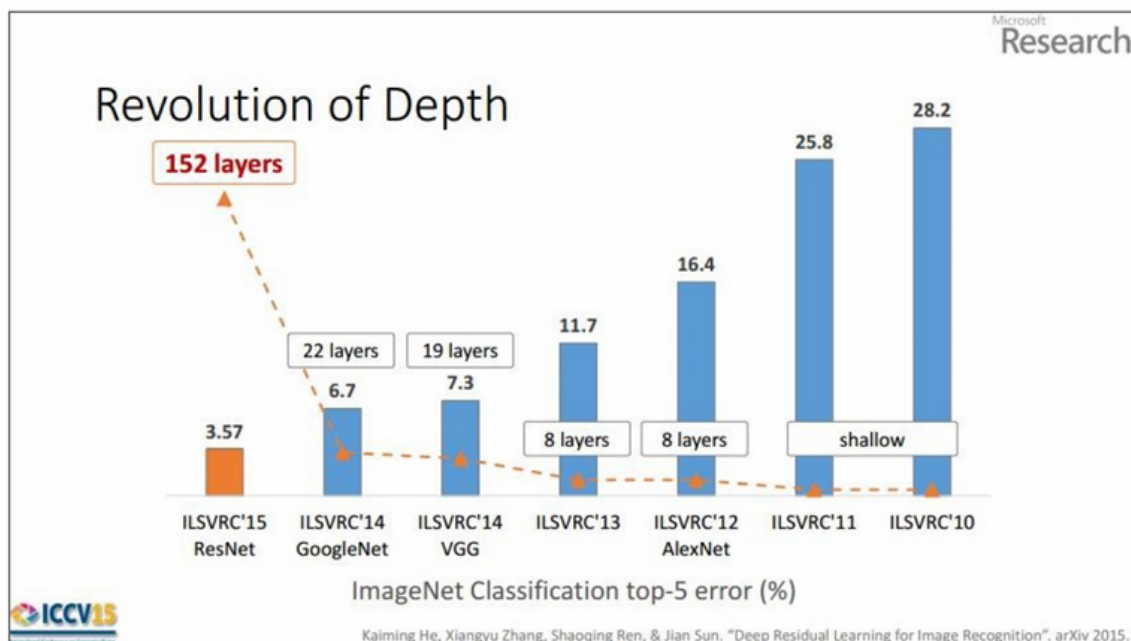
在ImageNet数据集上的top-5错误率比AlexNet低（6.67% vs. 15.4%）。

8. ResNet

由He等人在2015年提出。ResNet在ILSVRC 2015（ImageNet Large Scale Visual Recognition Challenge）比赛中取得了冠军，其top-5错误率为3.6%。

ResNet通过引入残差学习（residual learning）的概念，解决了深度网络训练中的退化问题，使得构建更深的网络成为可能，从而在多个视觉任务中取得了显著的性能提升。

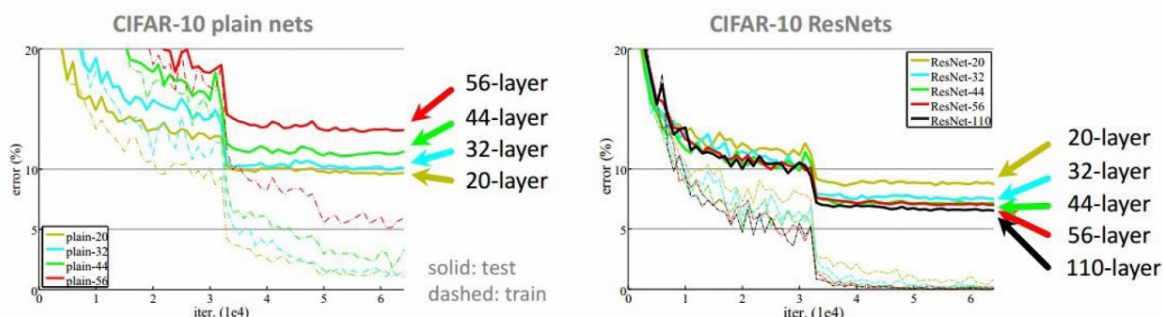
下图中列出了从2010年到2015年的几个主要CNN架构及其在top-5错误率（即前5个预测中正确分类的比例）上的表现。



每种架构旁边标注了其层数，例如AlexNet有8层，VGG有19层，GoogLeNet有22层，而ResNet有152层。

随着CNN架构的层数增加，模型在ImageNet分类任务上的性能显著提高，特别是ResNet通过引入残差学习解决了深度网络训练的难题，实现了更低的错误率。

下图显示了不同层数（20层、32层、44层、56层、110层）的ResNet在训练和测试过程中的错误率变化。

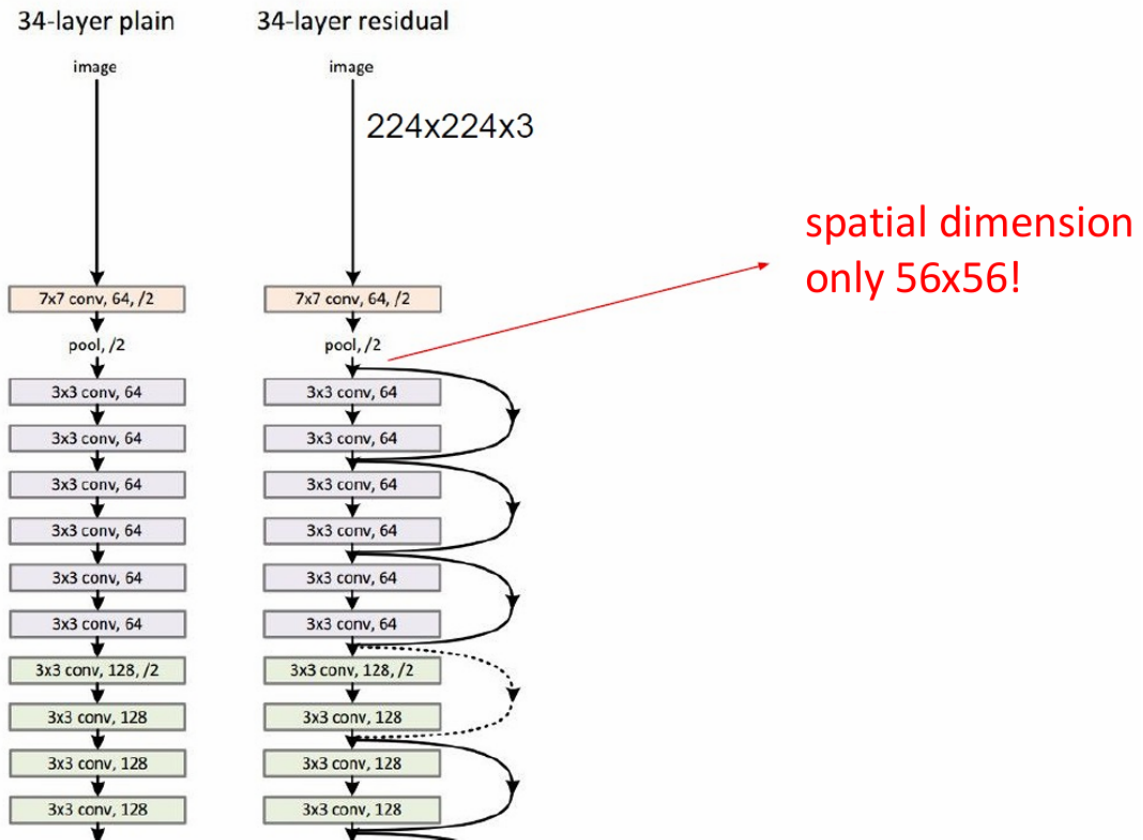


ResNet的错误率明显低于普通网络，特别是在网络层数较多时（例如56层和110层）。

随着层数的增加，ResNet的错误率持续下降，显示出更好的泛化能力和学习能力。

残差网络（ResNet）通过引入残差连接有效地解决了深度网络训练中的退化问题，使得可以构建更深的网络而不会遇到性能瓶颈。

此外esNet可以在2-3周内使用8个GPU机器完成训练。尽管ResNet的层数远多于AlexNet和VGG，但其训练效率仍然很高。这也是因为ResNet的残差连接设计使得网络更容易训练，从而在推理时也更高效。



左侧展示了一个34层的普通卷积网络架构。

每一层都包含卷积层 (conv) 和池化层 (pool) 。

随着网络深度的增加，特征图的空间维度逐渐减小。

右侧展示了一个34层的残差网络架构。

残差网络通过引入残差连接来解决深度网络中的退化问题，使得网络可以更深。

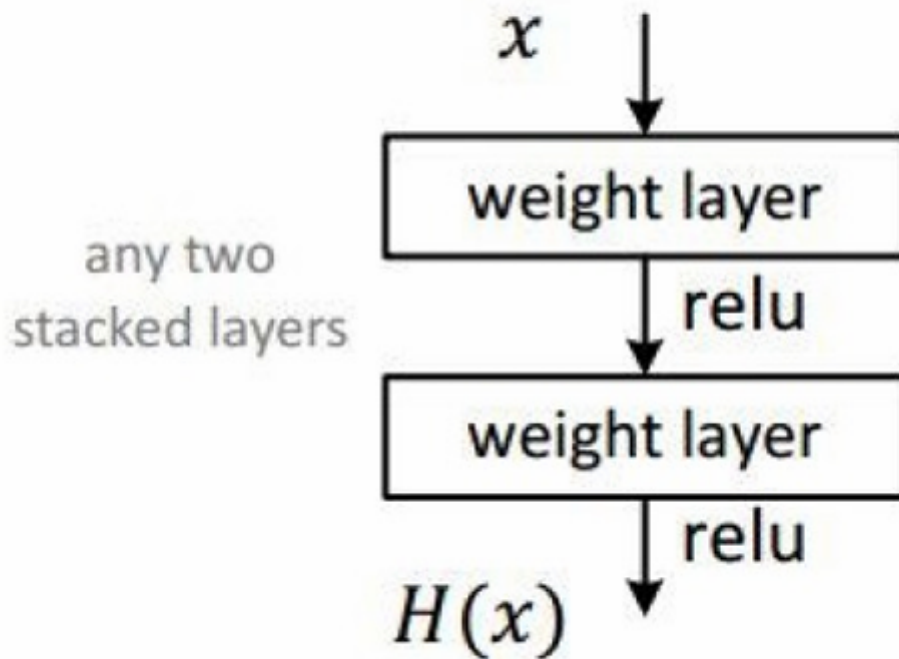
图中用箭头和虚线表示残差连接，这些连接跳过了一些层，直接将输入传递到后面的层。

尽管网络更深，但空间维度（宽度和高度）仅减小到56x56，而不是像普通网络那样减小到更小的尺寸。

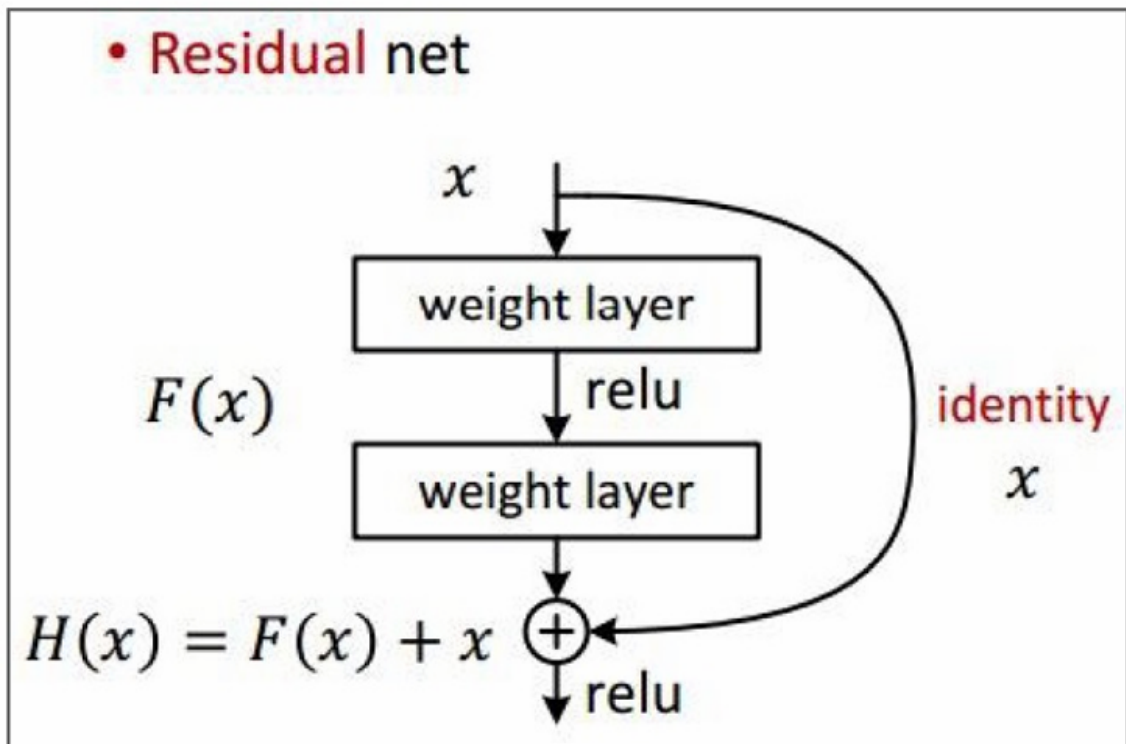
这种设计有助于保留更多的空间信息，从而提高网络的性能。

8.1 残差网络 (Residual net)

- Plaint net



普通的卷积神经网络结构，其中包含两个卷积层（weight layer）和ReLU激活函数（relu）。输入 x 经过第一个卷积层和ReLU激活后，得到 $F(x)$ ，然后经过第二个卷积层和ReLU激活得到最终输出 $H(x)$ 。



残差网络通过引入残差连接（identity shortcut connection）来解决深度网络训练中的退化问题。

输入 x 通过一个残差模块，该模块包含两个卷积层和ReLU激活，输出为 $F(x)$ 。

残差模块的输出 $F(x)$ 与输入 x 相加（通过残差连接），然后通过ReLU激活函数得到最终输出

$H(x) = F(x) + x$ 。

这种结构使得网络可以学习到输入和输出之间的残差 $F(x)$ ，而不是直接学习输出 $H(x)$ ，从而简化了学习过程。

这样的方式残差连接允许网络在训练过程中直接传递输入信息，这有助于缓解梯度消失问题，并使得网络能够更容易训练更深的层数。

9. 总结

CNN架构概述：

CNN通常由卷积层（CONV）、池化层（POOL）、全连接层（FC）堆叠而成。

这种架构的目的是提取图像特征并进行分类。

趋势：更小的滤波器和更深的架构：

近年来，研究者倾向于使用更小的卷积核（例如1x1或3x3）和更深的网络架构。

这有助于捕捉更精细的特征，同时减少参数数量。

去掉POOL/FC层的趋势：

传统的CNN架构中，POOL层用于降维，FC层用于分类。

但最新的研究（如ResNet和GoogLeNet）表明，可以去掉POOL和FC层，仅使用卷积层（CONV）。

这种趋势使得网络更简单，同时保持或提高性能。

典型架构：

典型的CNN架构形式为：[(CONV-RELU) N -POOL?] M -(FC-RELU)* K 。

其中，CONV表示卷积层，RELU表示ReLU激活函数，POOL表示池化层，FC表示全连接层。

N 通常为1到5， M 是一个较大的数字， K 是输出类别数（例如，对于ImageNet， $K=1000$ ）。

软最大池化（SOFTMAX）通常用于池化层。

ResNet/GoogLeNet的挑战：

ResNet和GoogLeNet通过引入残差连接和Inception模块，挑战了传统的CNN架构范式。

这些架构展示了即使没有POOL和FC层，网络也能有效地学习特征并进行分类。